

Análisis de Steam: Procesamiento Big data para sistemas de recomendación

Asignatura: Big data

Integrantes:

- Benjamín Lagos
- Vianca Viveros
- Andrés Chávez
- Danko Gangas

Sección: P04

Fecha de entrega: 13/09/2026

Contexto del caso

Steam, es una plataforma de distribución de videojuegos fundada por VALVE y ampliamente utilizada por los usuarios debido a su magnitud y comodidad. Steam desde sus inicios ha tenido un fuerte crecimiento en títulos de distintas distribuidoras desde su fundación en 2003, por lo que la cantidad de registros ha aumentado drásticamente como para ser manejable.

Dado esta premisa, existe la necesidad de implementar una arquitectura de BIG DATA basado en el ecosistema Hadoop, para poder manejar el número creciente de nuevos títulos que se publican cada año. Se busca encontrar los títulos con mejores reseñas para poder recomendar más fervientemente ciertos videojuegos.

Objetivo del proyecto

La solución que se implementa es con el objetivo de procesar el catálogo masivo de videojuegos que posee Steam, para así apuntar a aquellos con mejor recepción y reseñas por parte de la comunidad, permitiendo así generar un sistema de recomendaciones basada en datos concretos.

Descripción del Dataset

Se uso un archivo CSV de kaggle, el cual proviene de un conjunto de datos de juegos de Steam correspondientes al año 2025.

Volumen y tamaño: Contiene 94.948 registros y 47 columnas de variables por título, con un peso de archivo de 471 MB.

Variables relevantes:

- name: Nombre del juego.
- price: Precio del juego.
- peak_ccu: Máxima cantidad de jugadores simultáneos.
- average_playtime: Tiempo promedio de juego.
- pct_pos_total: Porcentaje de reseñas positivas.
- num_reviews_total: Volumen o cantidad de reseñas existentes.
- metacritic_score: Valoración de metacritic.

Estructura del archivo: La primera línea del archivo contiene y define los nombres de 47 variables. A partir de la fila dos, cada una de ellas corresponde a un único registro, (una fila por juego). Para las variables que contienen descripciones se usaron comillas dobles (“ ”) para así encapsular los textos.

Problemas de calidad: Al revisar los registros se encontraron cerca de 486.000 casillas vacías. La variable score_rank contenía 94.909 nulos, por lo que su uso para la clasificación fue descartado.

Necesidades de información

Para poder responder el objetivo de recomendar las mejores opciones de videojuegos, se definieron el uso de los siguientes parámetros:

- **Ranking de valoración:** Identificar juegos con altas valoraciones que posean un respaldo estadístico, por medio del volumen de reseñas y el porcentaje de positividad de estas.
- **Peak de jugadores:** Detectar los juegos con alta viralidad y con mayor cantidad de jugadores conectados de manera simultánea.
- **Criterio de gratuidad:** Descubrir los juegos gratuitos más recomendados por la comunidad y así atraer nuevos usuarios.
- **Crítica especializada:** Evaluar que títulos poseen el mejor puntaje de prensa dedicada, como metacritic.
- **Rentabilidad:** Medir que juegos son los que generan la mayor cantidad de tiempo en ellos con relación a su precio de venta.

Estructura de HDFS

Para asegurar el orden del proyecto, se diseñó una jerarquía de carpetas y directorios dentro del sistema de archivos distribuido (HDFS):

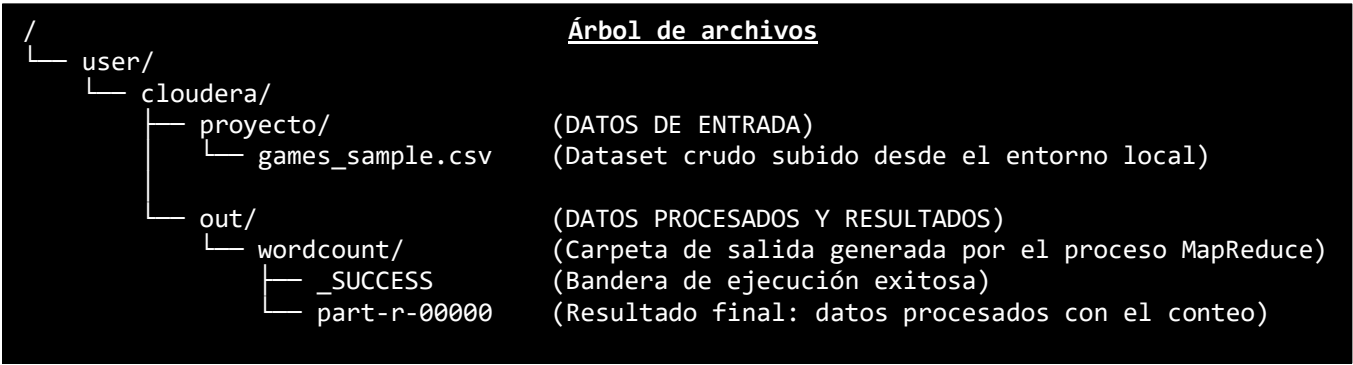


Figura 1 - Árbol de archivos del proyecto.

Para garantizar la trazabilidad e integridad de la información se separaron los ambientes de archivos:

- **user/cloudera/proyecto/:** Aquí van los datos de entrada de proyecto. Su función principal es almacenar el dataset en crudo. Para evitar cualquier tipo de corrupción de archivos o eliminación por error, se preservan los datos originales por separado.
- **user/cloudera/out/wordcount/:** Su principal función es alojar los datos ya procesados. Se creo ya que MapReduce requiere un directorio de salida exclusivo y vacío para cada trabajo que ejecuta, gracias a esto se previene la sobreescritura de datos accidental.

Evidencia 1: Directorio de entrada y subida de datos.

```
[cloudera@quickstart ~]$ hadoop fs -put ~/workspace/steam/* /user/cloudera/proyecto/
[cloudera@quickstart ~]$ hadoop fs -ls /user/cloudera/proyecto/
Found 1 items
-rw-r--r-- 1 cloudera cloudera      867469 2026-09-12 16:23 /user/cloudera/proyecto/games_sample.csv
[cloudera@quickstart ~]$
```

Figura 2 - Datos de entrada hacia el entorno

Transferencia del dataset de kaggle hacia la carpeta raíz del proyecto en HDFS mediante el comando -put.

Evidencia 2: Resultados ya cargados.

```
[cloudera@quickstart codigo]$ hadoop fs -cat /user/cloudera/out/wordcount/*
A          96
AB          4
ACCESSIBILITY  4
ACCLAIMED    3
ACTION       3
AETERNUM     4
AGAINST      2
AI           6
AIR          2
ALIEN        2
ALL          4
ALONGSIDE    2
ALSO         1
AMAZING      1
AN           4
AND          12
ANOTHER      4
ARE          2
ARK          43
ARKs         7
ARMED        2
ARPG         2
ARPGs        1
ARSENAL      2
Abducted     2
Abilities    4
Ability      1
```

Figura 3 - Datos procesados en el entorno.

Se verifica la salida de datos almacenados correctamente en la carpeta /out/wordcount.

Procesos desarrollados

1. Sqoop

Para realizar la importación de datos hacia el ecosistema Hadoop mediante Sqoop, se optó por emular la conexión a una base de datos relacional. Para ello, los registros del dataset de Steam se cargaron inicialmente en una base de datos MySQL local. Posteriormente, se ejecutó el comando de importación con Sqoop para transferir de manera íntegra la tabla hacia el sistema de archivos distribuido (HDFS), dejando la información disponible para las siguientes etapas de procesamiento.

```
sqoop import \  
--connect jdbc:mysql://127.0.0.1/proyecto_I \ --LOCALHOST--  
--username root \  
--password cloudera \  
--table steam_games \  
--m 1 \  
--target-dir /user/cloudera/steam_games_hdfs
```

Figura 4 - Comandos Sqoop de importación

2. MapReduce

El propósito de implementar MapReduce en este caso práctico es procesar de manera distribuida y eficiente el gran volumen de datos del catálogo de Steam, el cual cuenta con más de 90.000 registros. Se utiliza el algoritmo **WordCount** para analizar y contar la frecuencia de palabras dentro de los campos de texto del dataset, tales como categorías, etiquetas o descripciones. Esto permite extraer métricas cuantificables sobre los términos, características o géneros más predominantes en la plataforma, facilitando el análisis de tendencias dentro del ecosistema de videojuegos de forma escalable.

```
[cloudera@quickstart codigo]$ jar -cvf wordcountmapreduce.jar *.class  
added manifest  
adding: WordCountMapReduce.class(in = 1991) (out= 985)(deflated 50%)  
adding: WordCountMapReduce$Map.class(in = 2299) (out= 1036)(deflated 54%)  
adding: WordCountMapReduce$Reduce.class(in = 1654) (out= 689)(deflated 58%)  
[cloudera@quickstart codigo]$
```

*Figura 5 - Empaquetado de las clases Java
compiladas en un archivo ejecutable .jar.*

Resultado: El proceso se ejecutó correctamente en el clúster tomando como entrada el dataset original de Steam. Como resultado, MapReduce generó un archivo de salida consolidado en HDFS que contiene el recuento exacto de la frecuencia de términos, demostrando la aplicación del procesamiento distribuido sobre el conjunto de datos.

```
[cloudera@quickstart codigo]$ hadoop fs -cat /user/cloudera/out/wordcount_resultado/*
A          96
AB         4
ACCESSIBILITY  4
ACCLAIMED    3
ACTION    3
AETERNUM     4
AGAINST    2
AI          6
AIR         2
ALIEN       2
ALL         4
ALONGSIDE    2
ALSO        1
AMAZING     1
AN          4
AND        12
ANOTHER    4
ARE         2
ARK        43
ARKs       7
ARMED       2
ARPG        2
ARPGs       1
ARSENAL     2
Abducted    2
Abilities    4
Ability     1
About      43
Abyss       2
```

Figura 6 - Muestra de los resultados obtenidos tras la ejecución del proceso MapReduce en el clúster.

3. Pig

El propósito de utilizar Apache Pig en este proyecto es procesar, transformar y analizar el gran volumen de datos de Steam mediante Pig Latin, un lenguaje de scripting de alto nivel. Su uso permite realizar operaciones de agrupación, filtrado y conteo de una manera mucho más ágil y con menos líneas de código en comparación con la escritura nativa de MapReduce en Java, facilitando la extracción rápida de métricas sobre los videojuegos.

```
2026-09-13 12:00:53,395 [main] INFO org.apache.pig.backend.hadoop.executionengine.mapReduceLayer.MapReduceLauncher - Success!
```

Figura 7 - Mensaje de confirmación de ejecución exitosa del script wordcount.pig.

Resultado: Como resultado de la ejecución, el script de Pig procesó exitosamente la información y almacenó un nuevo conjunto de datos consolidados en HDFS. La salida evidencia la agrupación de los videojuegos según su precio y contabiliza la frecuencia exacta de cada valor dentro del catálogo procesado. Este recuento permite extraer métricas directas, como la existencia de 30 títulos gratuitos (valor 0.0), entregando información

estructurada para abordar los criterios de gratuidad y rentabilidad definidos en las necesidades del proyecto.

```
[cloudera@quickstart wordcount]$ hadoop fs -cat /user/cloudera/out/pig_resultado/*
0.0,30
19.99,16
59.99,12
29.99,9
39.99,7
9.99,5
14.99,5
49.99,4
24.99,3
4.99,2
22.49,1
13.99,1
29.24,1
5.99,1
3.99,1
2.99,1
price,1
```

Figura 8 - Archivo de salida de Apache Pig en HDFS, detallando la frecuencia de juegos por precio.

4. Oozie

El propósito de utilizar Apache Oozie en este caso práctico es orquestar y automatizar el flujo de trabajo (workflow) del procesamiento de datos de Steam. Oozie permite programar y gestionar la ejecución de las tareas (en este caso, el script de Pig que cuenta los precios) de manera centralizada y secuencial dentro del clúster de Hadoop, evitando la ejecución manual y asegurando la correcta coordinación de las herramientas del ecosistema.

```
[cloudera@quickstart wordcount]$ oozie job -oozie http://localhost:11000/oozie -config job.properties -run
job: 00000004-260912054643352-oozie-oozi-W
```

Figura 9 - Ejecución del flujo de trabajo en Apache Oozie y generación del ID de trabajo.

Resultado: Como resultado de la orquestación, Oozie administró y ejecutó de manera exitosa el flujo de trabajo programado (workflow). La verificación del estado final del trabajo confirma que la integración entre las herramientas del ecosistema operó sin errores, logrando automatizar el procesamiento de los datos de Steam de forma centralizada y eficiente.

```
[cloudera@quickstart wordcount]$ oozie job -oozie http://localhost:11000/oozie -info 0000004-260912054643352-oozie-oozi-W
Job ID : 0000004-260912054643352-oozie-oozi-W
-----
Workflow Name : pig-steam-wf
App Path      : hdfs://quickstart.cloudera:8020/user/cloudera/oozie_job
Status        : SUCCEEDED
Run           : 0
User          : cloudera
Group         : -
Created       : 2026-09-13 20:59 GMT
Started       : 2026-09-13 20:59 GMT
Last Modified : 2026-09-13 21:01 GMT
Ended        : 2026-09-13 21:01 GMT
CoordAction ID: -

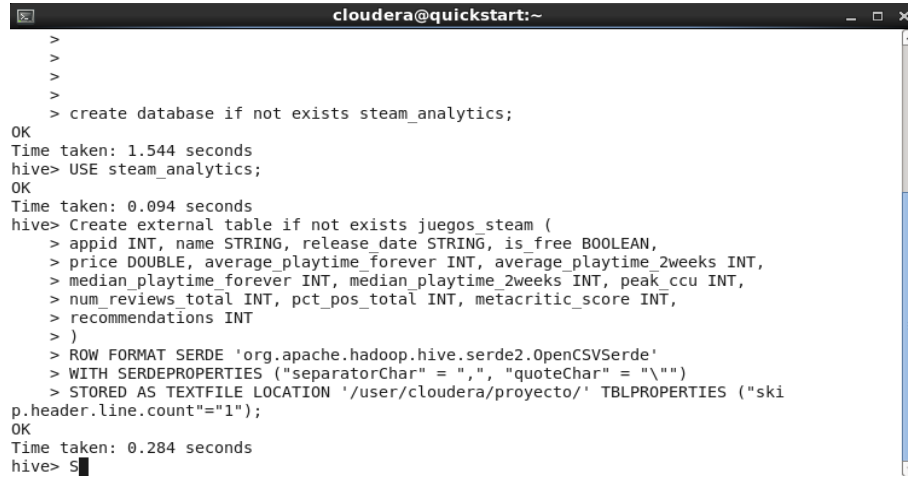
Actions
-----
ID                                     Status  Ext ID                               Ext Status Err Code
-----
0000004-260912054643352-oozie-oozi-W@start:      OK      -                                     OK          -
0000004-260912054643352-oozie-oozi-W@pig-node     OK      job_1789217126145_0010 SUCCEEDED  -
0000004-260912054643352-oozie-oozi-W@end         OK      -                                     OK          -
-----
```

Figura 10 - Confirmación de ejecución exitosa (SUCCEEDED) del flujo de trabajo orquestado por Oozie.

5. Hive e Impala

El propósito de implementar Hive e Impala en conjunto es habilitar un entorno de consultas estructuradas mediante SQL sobre los datos almacenados en HDFS. Hive se utiliza para definir los esquemas, metadatos y aplicar formatos complejos de lectura, mientras que Impala se emplea para realizar consultas analíticas de manera interactiva y con tiempos de respuesta mucho más rápidos gracias a su procesamiento en memoria.

Primero, se creó la base de datos analítica steam_analytics y la tabla externa juegos_steam utilizando Hive. Debido a que el dataset de Steam contiene descripciones con comas internas, se aplicó la librería OpenCSVSerde en la estructuración de la tabla para garantizar que las columnas no se desplazaran y los datos mantuvieran su integridad original. Posteriormente, para sincronizar los motores, se accedió a la consola de Impala y se ejecutó el comando INVALIDATE METADATA;, lo que permitió actualizar el metastore y lograr que Impala reconociera de inmediato la tabla y los datos definidos en Hive.



```
cloudera@quickstart:~  
>  
>  
>  
>  
> create database if not exists steam_analytics;  
OK  
Time taken: 1.544 seconds  
hive> USE steam_analytics;  
OK  
Time taken: 0.094 seconds  
hive> Create external table if not exists juegos steam (  
  > appid INT, name STRING, release_date STRING, is_free BOOLEAN,  
  > price DOUBLE, average_playtime_forever INT, average_playtime_2weeks INT,  
  > median_playtime_forever INT, median_playtime_2weeks INT, peak_ccu INT,  
  > num_reviews_total INT, pct_pos_total INT, metacritic_score INT,  
  > recommendations INT  
  > )  
  > ROW FORMAT SERDE 'org.apache.hadoop.hive.serde2.OpenCSVSerde'  
  > WITH SERDEPROPERTIES ("separatorChar" = ",", "quoteChar" = "\"")  
  > STORED AS TEXTFILE LOCATION '/user/cloudera/proyecto/' TBLPROPERTIES ("ski  
p.header.line.count"="1");  
OK  
Time taken: 0.284 seconds  
hive> S
```

Figura 11 - Evidencia de creación de metadatos en

Como resultado de este proceso, el entorno quedó configurado, estructurado y sincronizado correctamente en ambos motores. La información del catálogo de Steam quedó disponible para ser interrogada mediante consultas SQL de alto rendimiento, logrando extraer métricas cuantificables. La aplicación de estas sentencias y el análisis del negocio derivado de ellas se exponen en la sección de **Consultas y resultados**.

6. Flume

El propósito de implementar Apache Flume es automatizar la ingesta y recolección de datos hacia el sistema de archivos distribuido (HDFS). En este proyecto, su función es capturar y transferir de forma eficiente los registros del catálogo de Steam desde un directorio local hacia el clúster, simulando un flujo de entrada que consolida la información para que sea procesada posteriormente por el resto de las herramientas del ecosistema.

Para la ingesta, se diseñó un agente de Flume compuesto por un Source tipo *spooldir* (que monitorea una carpeta local en busca de archivos nuevos), un Channel en memoria para agilizar la transferencia, y un Sink configurado para escribir los datos de manera persistente en HDFS.

```
[cloudera@quickstart wordcount]$ flume-ng agent --name a1 --conf conf --conf-file flume.conf -Dflume.root.logger=INFO,console
Info: Including Hadoop libraries found via (/usr/bin/hadoop) for HDFS access
Info: Including HBASE libraries found via (/usr/bin/hbase) for HBASE access
Info: Including Hive libraries found via () for Hive access
```

Figura 12 – Inicialización del agente de Flume en consola

Resultado: El agente de Flume procesa los registros del archivo original y los transfiere hacia el sistema de archivos distribuido de Hadoop. Como resultado de esta ingesta, la información cruda del catálogo de Steam queda centralizada, estructurada y lista para ser consumida por las herramientas de procesamiento y consulta del ecosistema, como MapReduce y Hive.

```
[cloudera@quickstart wordcount]$ hadoop fs -ls /user/cloudera/proyecto/
Found 1 items
-rw-r--r--  1 cloudera cloudera    867469 2026-09-12 16:23 /user/cloudera/proyecto/games_sample.csv
```

Figura 13 – Verificación de ingesta de datos en HDFS

Consultas y Resultados

Para generar el análisis del negocio se utilizaron las siguientes consultas mediante Hive e Impala:

1. Valida qué juegos mantienen el mayor porcentaje de reseñas positivas respaldadas por un alto volumen de reseñas (más de 1.000 reseñas válidas).

```
SELECT name, pct_pos_total, num_reviews_total FROM juegos_steam WHERE num_reviews_total > 1000 ORDER BY pct_pos_total DESC, num_reviews_total DESC LIMIT 5;
```

2. Revela los títulos que poseen la mayor cantidad de jugadores simultáneos históricamente.

```
SELECT name, peak_ccu, price FROM juegos_steam ORDER BY peak_ccu DESC LIMIT 5;
```

3. Localiza productos accesibles sin costo alguno (gratis), que lograron alta viralidad y recomendación en la comunidad.

```
SELECT name, recommendations, num_reviews_total FROM juegos_steam WHERE price = 0 ORDER BY recommendations DESC LIMIT 5;
```

4. Evalúa la concordancia entre la valoración de la prensa especializada (Metacritic) y la positividad de los usuarios reales.

```
SELECT name, metacritic_score, pct_pos_total FROM juegos_steam WHERE metacritic_score > 0 ORDER BY metacritic_score DESC LIMIT 5;
```

5. Determina qué juegos devuelven mayor cantidad de horas de entretenimiento de los usuarios en relación con su precio.

```
SELECT name, average_playtime_forever, price FROM juegos_steam WHERE average_playtime_forever > 0 ORDER BY average_playtime_forever DESC LIMIT 5;
```

Resultado Consulta 1

Hive

Add a name... Add a description...

36.24s test1 text ?

```
1 SELECT name, pct_pos_total, num_reviews_total
2 FROM juegos_steam
3 WHERE num_reviews_total > 1000
4 ORDER BY pct_pos_total DESC, num_reviews_total DESC LIMIT 5;
```

Query History Saved Queries Results (5) Q ✓

	name	pct_pos_total	num_reviews_total
1	Rogue Tower	140	5366
2	Yet Another Zombie Survivors	115	9493
3	Master of Orion	102	7032
4	Spellbook Demonslayers	100	1195
5	Hand of Fate 2	99	5121

Impala

Add a name... Add a description...

3.72s test1 text ?

```
1 SELECT name, pct_pos_total, num_reviews_total
2 FROM juegos_steam
3 WHERE num_reviews_total > 1000
4 ORDER BY pct_pos_total DESC, num_reviews_total DESC LIMIT 5;
```

Query History Saved Queries Results (5) Q ✓

	name	pct_pos_total	num_reviews_total
1	Helltaker	NULL	2780
2	The Occupation	NULL	1987
3	Bahnsen Knights	NULL	1986
4	Exploration Pro	512.00	1024
5	Rogue Tower	140.00	5366

Resultado consulta 2

Hive

Add a name...

Add a description...

2m, 17s

test1

text

1

SELECT name, peak_ccu, price

2

FROM juegos_steam

3

ORDER BY peak_ccu DESC LIMIT 5;

Query History

Q

Saved Queries

Q

Results (5)

Q

<

Resultado consulta 3

Hive

Add a name...

Add a description...

1m, 9s

test1

text

```
1 SELECT name, recommendations, num_reviews_total
2 FROM juegos_steam
3 WHERE price = 0
4 ORDER BY recommendations DESC LIMIT 5;
```

Query History

Saved Queries

Results (5)

	name	recommendations	num_reviews_total
1	Half-Life 2: Deathmatch	9427	NULL
2	Retroninjacyberassassin	6617	NULL
3	The 52 Fragments	2023	NULL
4	Dead Abyss	1977	0
5	Wwbit	1122	NULL

Impala

Add a name...

Add a description...

4.78s

test1

text

```
1 SELECT name, recommendations, num_reviews_total
2 FROM juegos_steam
3 WHERE price = 0
4 ORDER BY recommendations DESC LIMIT 5;
```

Query History

Saved Queries

Results (5)

	name	recommendations	num_reviews_total
1	Counter-Strike 2	NULL	NULL
2	Team Fortress 2	NULL	NULL
3	Empires Mod	NULL	NULL
4	EVE Online	NULL	NULL
5	Dota 2	NULL	NULL

Resultado Consulta 4

Hive

Add a name... Add a description...

24.57s test1 text ?

1 SELECT name, metacritic_score, pct_pos_total

2 FROM juegos_steam

3 WHERE metacritic_score > 0

4 ORDER BY metacritic_score DESC LIMIT 5;

Query History Saved Queries Results (5) Q v

	name	metacritic_score	pct_pos_total
1	Warrior Adventure	12345678	NULL
2	Gay World	2269	NULL
3	Doom In the Dark	2025	0
4	NEOTAG LEAGUE	2025	NULL
5	Batlow - Mages And Monsters	2024	NULL

Impala

Add a name... Add a description...

3.1s test1 text ?

1 SELECT name, metacritic_score, pct_pos_total

2 FROM juegos_steam

3 WHERE metacritic_score > 0

4 ORDER BY metacritic_score DESC LIMIT 5;

Query History Saved Queries Results (5) Q v

	name	metacritic_score	pct_pos_total
1	Warrior Adventure	12345678	NULL
2	Gay World	2269	NULL
3	NEOTAG LEAGUE	2025	NULL
4	Doom In the Dark	2025	0.00
5	Batlow - Mages And Monsters	2024	NULL

Resultado Consulta 5

Hive

Add a name... Add a description...

30.45s test1 text ?

1 SELECT name, average_playtime_forever, price

2 FROM juegos_steam

3 WHERE average_playtime_forever > 0

4 ORDER BY average_playtime_forever DESC LIMIT 5;

Query History Saved Queries Results (5) Q v

	name	average_playtime_forever	price
1	Rage Melee	7926354432	0
2	Platformer Slider	123456792	0.99
3	Counter-Strike 2	4401570	0
4	Warhammer 40	112107	0
5	Need for Speed™ Heat	105010	69.99

Impala

Add a name... Add a description...

0.69s test1 text ?

1 SELECT name, average_playtime_forever, price

2 FROM juegos_steam

3 WHERE average_playtime_forever > 0

4 ORDER BY average_playtime_forever DESC LIMIT 5;

Query History Saved Queries Results (5) Q v

	name	average_playtime_forever	price
1	Rage Melee	7926354432	0.00
2	Platformer Slider	123456792	0.99
3	Counter-Strike 2	4401570	0.00
4	Warhammer 40	112107	0.00
5	Need for Speed™ Heat	105010	69.99

Conclusión

La implementación de esta arquitectura Big Data demostró ser altamente efectiva para procesar, transformar y analizar el catálogo masivo de Steam de manera escalable. A nivel de resultados, se logró consolidar un ecosistema Hadoop completamente funcional: HDFS actuó como un repositorio central robusto desde la ingesta inicial (mediante SCP y Flume), MapReduce y Pig procesaron de manera distribuida el conteo de términos y la agrupación de precios, Oozie orquestó el flujo de trabajo automatizado, y la dupla Hive/Impala habilitó el análisis interactivo mediante SQL. Esto permitió dar respuesta directa a las necesidades del negocio, extrayendo métricas concretas sobre gratuidad, horas de juego, rentabilidad y viralidad para alimentar el sistema de recomendaciones.

Durante el desarrollo, las dificultades que enfrentamos como equipo abarcaron distintas capas de la arquitectura. En la fase de preparación, la interoperabilidad entre Windows y el entorno Linux generó bloqueos de seguridad en los túneles SSH y transferencias SCP. En la etapa de procesamiento, los scripts de Pig fallaron por desajustes estructurales en los índices de columnas y dependencias desactualizadas. Sin embargo, los mayores desafíos surgieron en la orquestación con Oozie: un bucle infinito de conexión (estado PREP) debido a una mala configuración del puerto de YARN, caídas abruptas (estado KILLED) por la estricta regla de Hadoop de no sobrescribir carpetas de salida existentes, y fallos por la ausencia de librerías en el clúster. Además, a nivel de datos, las comas internas en las descripciones de los juegos desplazaban las columnas, rompiendo la estructura de las tablas.

Para solucionar estos bloqueos, se forzó el uso de protocolos heredados para asegurar la transferencia inicial de archivos. En Pig, se optimizó el código eliminando sentencias de impresión en pantalla (DUMP) para evitar la saturación de memoria del clúster. Para estabilizar Oozie, se apuntó correctamente al puerto 8032, se implementó una limpieza manual de directorios en HDFS y se inyectó el parámetro `oozie.use.system.libpath=true` para forzar la carga de dependencias. Finalmente, para mantener la integridad de los datos en Hive, se optó por implementar la librería `OpenCSVSerde`, y se ejecutó el comando `INVALIDATE METADATA` para sincronizar los esquemas inmediatamente con Impala.

Para oportunidades de mejora para el futuro, se propone evolucionar este ecosistema hacia un *pipeline* de datos de ingesta continua, automatizando el flujo completo desde la captura hasta el análisis sin intervención manual. Asimismo, el siguiente paso lógico sería conectar el motor de alta velocidad de Impala a una herramienta de Business Intelligence (como Power BI o Tableau) para transformar las consultas SQL actuales en paneles visuales e interactivos en tiempo real, facilitando la toma de decisiones estratégicas de forma más ágil.